

مراجعة مشروع Book Social Network + أسئلة الإنترفيو

المشروع: <https://github.com/EsmaelSamir/book-social-network>، والنسخة التي اتراجعت هي commit 078d813 (بتاريخ 11 يوليو 2026).

اتعمل إليه في المراجعة: اتقرا كل كود الباك إند والفرونٲ إند، واتعمل build للاتنين، واتشغلت الـ tests. بعد كده الباك إند اشتغل فعلاً على PostgreSQL واتجرب بطلبات حقيقية (curl)، والفرونٲ اتجرب في متصفح (Playwright). أي غلط جنبه ✓ معناه إنه اتشاف وهو يحصل فعلاً في التجربة، مش مجرد استنتاج من قراية الكود. مسارات ملفات الباك إند كلها تحت book-network/src/main/java/com/global/book/، والفرونٲ تحت book-network-ui/src/app/.

الفهرس

1. الخلاصة: المشروع تمام ولا لأ؟
2. الأخطاء بالترتيب (من الأخطر للأبسط)
3. أسئلة الباك إند (Spring Boot)
4. أسئلة الفرونٲ إند (Angular)
5. لو طلبوا تعديل أو Live coding
6. حاجات تانية مهمة قبل الإنترفيو
7. أرقام ومعلومات لازم تبقى محفوظة

1. الخلاصة: المشروع تمام ولا لأ؟

الفكرة والشكل العام كويسين جدًا كمشروع Full-stack للإنترفيو، والمشروع يغطي حاجات كتير بيحبوا يسألوا فيها: Spring Boot 4 و Spring Security مع JWT و JPA/PostgreSQL وإيميلات Async و Swagger، و Angular 21 بـ standalone components و lazy loading و guard و interceptor، و Docker و CI.

- الباك إند بيعمل build من غير مشاكل، وال 13 test كلهم ناجحين ✓.
- الفرونت بيعمل build ✓، بس فيه warning إن حجم الـ bundle أكبر من الـ budget.

بس فيه أخطاء حقيقية لازم تتصلح قبل ما المشروع يتعرض في إنترفيو:

- 4 مشاكل خطيرة:
 - أي يوزر يقدر يعدّل كتب غيره ويغيّر صورها (IDOR).
 - الـ JWT secret مرفوع على GitHub، واتعمل بيه token مزور ودخل فعلاً.
 - نفس الكتاب ممكن يتسلف لشخصين في نفس اللحظة.
 - مشكلة كبيرة في الفرونت: معظم الشاشات مش بتعرض الداتا بعد ما توصل (Angular 21 شغال zoneless والكود مكتوب بالطريقة القديمة). يعني لسته الكتب بتظهر فاضية، ورسالة غلط الـ login مش بتظهر، ونتيجة التفعيل مش بتظهر، وفورم التعديل بيظهر فاضي.
 - أخطاء تانية بتطلّع 500 في حالات عادية، زي إضافة كتاب من غير "Share me"، أو feedback من غير تقييم.
- الحلول أغلبها صغيرة. في حدود يوم شغل المشروع يبقى قوي جدًا، والأحسن كمان إن الأخطاء اللي هتتصلح تتحكي في الإنترفيو كقصص ("لقيت مشكلة كذا وحليتها إزاي")، وده يفرق جدًا.

2. الأخطاء بالترتيب

● خطيرة (Security / Data)

1) أي يوزر يقدر يعدّل كتاب يوزر ثاني (IDOR) ✓

- **فين:** 39-48 في `book/BookService.java` في `save()` ، في الجزء الخاص بال `update`.
- **اللي بيحصل:** لو اتبعث `POST /books` ومعاها `id` كتاب مش بتاع اليوزر، السيرفر بيعدّله عادي. في التجربة: Bob غيّر عنوان كتاب Alice لـ `HACKED BY BOB` والرد كان `200`. وكمان لو الكتاب مش موجود بيرمي `RuntimeException` فيبرجع 500 بدل 404.
- **الحل:** بعد `findById` يتعمل `check` إن `book.getOwner().getId()` هو نفس `id` اليوزر الحالي، وإلا يترمي `OperationNotPermittedException`. الكود في الجزء 5، تعديل 1.

2) أي يوزر يقدر يغيّر صورة غلاف كتاب غيره (IDOR) ✓

- **فين:** 205-214 في `book/BookService.java` في `uploadBookCoverPicture()`.
- **اللي بيحصل:** Bob رفع صورة لكتاب Alice والرد كان `202`، والصورة اتحفظت في فولدر Bob.
- **الحل:** نفس `check` الملكية.

3) الـ JWT secret وباسوورد الداتايز والميل مكتوبين في الكود ومرفوعين على GitHub ✓

- **فين:** `application-dev.yml` و `application-prod.yml` ، ونفس الـ `secret` في الاثنين.
- **اللي بيحصل:** اتعمل `token` مزوّر لـ `alice@test.com` بـ Python، باستخدام الـ `secret` اللي في الريبو ومن غير الباسوورد، والسيرفر قبله ورجّع `200`. يعني أي حد شاف الريبو يقدر يدخل بأي أكونت على أي سيرفر شغال بالكونفيج ده.
- **الحل:**
 - الـ `secret` يتقري من `environment variable`، يعني `secret-key: ${JWT_SECRET}`.
 - يبقى فيه `secret` مختلف لكل بيئة.
 - الـ `secret` الحالي يتغيّر (`rotate`)، لأنه بقى معروف.
 - يتضاف ملف `env.example` من غير قيم حقيقية.

4) نفس الكتاب ممكن يتسلف لاتنين في نفس اللحظة (Race condition) ✓

- **فين:** 136-164 في `book/BookService.java` في `borrowBook()`. الكود بيعمل `check` (`isAlreadyBorrowed`) وبعدين `save` ، من غير `transaction` ولا `lock`.
- **اللي بيحصل:** Bob و Carol طلبوا نفس الكتاب في نفس اللحظة 5 مرات، و4 مرات من الـ 5 الكتاب اتسلف للاتنين.
- **الحل:** أي واحد من دول (الكود في الجزء 5، تعديل 12):
 - إضافة `@Transactional` مع `@Lock(PESSIMISTIC_WRITE)` على قراءة الكتاب.
 - استخدام `@Version` (`optimistic locking`).
 - عمل `unique partial index` في الداتايز على `book_id` لما `return_approved = false`.

● أخطاء في الشغل (Bugs)

5) الشاشات مش بتتحدث بعد ما الداتا توصل (Angular zoneless) ✓ ده أكبر غلط في الفرونت

- **السبب:** Angular 21 بيشتغل **zoneless** افتراضياً، ومفيش `zone.js` في `package.json`. والكود مكتوب بالطريقة القديمة: قيمة بتتحط في field عادي بعد `await` أو `.then()`، زي `this.bookResponse = await ...`. فـ Angular مش بيعرف إن حاجة اتغيرت، ومش بيعيد رسم الشاشة.
- **اللي اتشاف في المتصفح:**
 - صفحة `/books` : ال component فيه 5 كتب، والشاشة فيها 0 كروت. أول ما change detection اشتغل بالعافية، ال 5 ظهوروا.
 - صفحة Login بباسوورد غلط: رسالة الغلط موجودة في ال component بس **مش ظاهرة**.
 - صفحة التفعيل: الأكونت **اتفعل فعلاً في الداتايز** بس رسالة "Activation Successful" مش بتظهر، فاليزور فاكرو إن مفيش حاجة حصلت.
 - صفحة تعديل كتاب (`/books/manage/2`) : الداتا وصلت، بس الفورم **ظاهر فاضي**.
 - صفحة Borrowed books: الجدول فاضي.
 - يعني الشاشة بتتحدث بس لما اليزور يعمل click على حاجة، فال UI دايمًا "متأخر خطوة".
- **الحل:**

- **الأحسن:** استخدام signals، يعني `bookResponse = signal<PageResponseBookResponse>({})` وبعدين `this.bookResponse.set(...)` وفي ال template يبقى `bookResponse().content` (الكود في تعديل 5).
- **حل سريع:** `inject(ChangeDetectorRef).markForCheck()` بعد كل تحديث.
- **أو:** رجوع `zone.js`، يعني `provideZoneChangeDetection()` مع ال `polyfill`.

6) صفحة التسجيل بتنقل لصفحة التفعيل حتى لو التسجيل فشل ✓

- **فين:** `pages/register-component/register-component.ts:29`. الاستدعاء `this.api.invoke(...)` معمول **من غير** `await`، فال `try/catch` ملوش أي لازمة.
- **اللي بيحصل:** اتجرب إيميل غلط وباسوورد 3 حروف، السيرفر رجّع 400، بس الصفحة راحت `/activate-account` ومفيش أي رسالة غلط.
- **الحل:** `async register()` مع `await`، ومعها signals عشان الرسالة تظهر.

7) إضافة كتاب من غير ما "Share me" يتعلم عليه بترجع 500 ✓

- **السبب:** حاجتين مع بعض:
 - الفورم مش بيعت `shareable` لو ال `checkbox` متملمشش، لأن `shareable: true` معمولها comment في `manage-book-component.ts:25`.
 - في الباك، `BookRequest.shareable` نوعه `Boolean` (wrapper)، فيوصل `null`، ويحصل `NullPointerException` وقت ال `unboxing` في `book/BookMapper.java:15`.
- **اللي بيحصل:** اتجرب من ال API ومن ال UI، والرد كان 500 برسالة `. Cannot invoke "java.lang.Boolean.booleanValue()"`

- **الحل:** في ال record يبقى boolean (primitive)، أو يتكتب Boolean.TRUE.equals(request.shareable()) . وفي Angular تبقى القيمة الابتدائية shareable: false .

8) Feedback من غير تقييم (note) يبيّض صفحة الكتب لكل الناس ✓

- **السبب:** FeedbackRequest.note مفيهوش @NotNull ، وال @Positive/@Min/@Max يبعّدوا ال null عادي. و Book.getRates() يعمل mapToDouble(Feedback::getNote) ، فيحصل NPE .
- **اللي بيحصل:** اتخط feedback واحد من غير note على كتاب، وبعدها GET /books و GET /books/owner و GET /books/{id} كلهم رجعوا 500 لكل اليوزرز.
- **الحل:** @NotNull على note و nullable = false على العمود. والأحسن كمان إن المتوسط يتحسب في الداتايز (AVG) .

9) لو صاحب الكتاب عمله Archive وهو متسلف، الكتاب يعلق للأبد ✓

- **فين:** returnBorrowedBook و approveReturnBorrowedBook بيرفضوا لو الكتاب archived أو مش shareable .
- **اللي بيحصل:** Bob استلف الكتاب، وبعدين Alice عملته Bob . archive. Bob مش عارف يرجّعه (400)، و Alice مش عارفة توافق على الإرجاع (400).
- **الحل:** الشرط ده يتشال من return و approve ، ويتمنع ال archive أو إلغاء ال share لو الكتاب متسلف.

10) رقم الصفحة في لسته ال Feedback غلط ✓

- **فين:** feedback/FeedbackService.java:56 ، مستخدم getNumberOfElements() بدل getNumber() .
- **اللي بيحصل:** الصفحة رقم 0 وفيها 3 تعليقات رجعت 3 : "number" .

11) التسجيل بإيميل موجود قبل كده بيرجع 500 ويطلّع ال SQL للعميل ✓

- **السبب:** مفيش check قبل الحفظ. الرد فيه اسم ال constraint وجمله ال INSERT كاملة.
- **الحل:**
 - استخدام findByEmail قبل الحفظ، ولو موجود يرجع **Conflict 409** برسالة واضحة.
 - إضافة handler ل DataIntegrityViolationException ، عشان لو طليين وصلوا في نفس اللحظة.

12) Error handling بيحوّل أي غلط ل 500 ويكشف تفاصيل داخلية ✓

- ال @ExceptionHandler(Exception.class) في handler/GlobalExceptionHandler.java:63 ييمسك كل حاجة، حتى أخطاء Spring العادية:

الطلب	المفروض يرجع	اللي رجع فعلاً
GET /books/999999 (مش موجود)	404	500
GET /books/abc	400	500
GET /does-not-exist	404	500
طلب JSON بايظ	400	500
كود تفعيل غلط	400	500 (والرسالة "Invalid Token" فيها typo)
رفع صورة 12MB	413	500

وكمان `exp.getMessage()` بيترجع للعميل، و `printStackTrace()` مستخدم بدل `logger`.

الحل:

- يتعمل handler لكل نوع: `EntityNotFoundException` → 404، و `HttpMessageNotReadableException` و `MethodArgumentTypeMismatchException` → 400، و `MaxUploadSizeExceededException` → 413.
- أو الكلاس يعمل `extend` لـ `ResponseEntityExceptionHandler`، ويستخدم `ProblemDetail`.
- الـ 500 يرجع رسالة عامة بس، والتفاصيل تتكتب بـ `log.error`.

13) من غير token، أو لو الـ token انتهى، الرد 403 فاضي بدل 401 ✓

- **السبب:** مفيش `AuthenticationEntryPoint` متعرف، والـ `JwtAuthFilter` مش بيمسك `ExpiredJwtException` ولا أي `JwtException`.
- **النتيجة:** الفرونت مش بيقدر يفرق بين "الجلسة انتهت" و "ممنوع".
- **الحل:**

- إضافة `exceptionHandling(e -> e.authenticationEntryPoint(new HttpStatusEntryPoint(HttpStatus.UNAUTHORIZED)))`.
- إضافة `try/catch` في الفلتر.
- إضافة `interceptor` في Angular يعمل `logout` لما يوصله 401 (تعديل 11).

14) لو الإيميل فشل محدش بيعرف، ومفيش طريقة لطلب كود جديد ✓

- **اللي بيحصل:**
 - الميثود `sendEmail` معموله `@Async`، فالغلط بيتسجل في اللوج بس، والتسجيل بيرجع 201 عادي. اتجرب والـ SMTP مقفول، وده اللي حصل بالظبط.
 - اليوزر مش هيقدر يسجل تاني لأن الإيميل `unique`، ومفيش `endpoint` لـ "resend code".
- **كمان:** إعدادات الميل في الـ YAML متخطوطة في مكان غلط، فكلها متجاهلة:
 - `mail.auth` بدل `mail.smtp.auth`
 - `mail.starttls.enabled` بدل `mail.smtp.starttls.enable`
 - الـ `timeouts` مكتوبة من غير `smtp`، واللوج يقول `1 - timeout`، يعني الانتظار مالوش آخر لو سيرفر الميل علّق.

15) كود التفعيل ضعيف

- الكود 6 أرقام بس (مليون احتمال) ومفيش rate limiting، ومش مربوط بالإيميل في الريكوس.
- عمود token **مش unique ومفيش عليه index**، وده اتشاف في الداتايز. لو اتنين أخذوا نفس الكود، findByToken هيرمي IncorrectResultSizeDataAccessException ويرجع 500.
- الكود بيشتغل تاني بعد ما يتستخدم، لأن مفيش check على validatedAt. ✓

16) حد رفع الصور الحقيقي 10MB مش 50MB ✓

- الإعداد max-file-size: 50MB متظبط، بس max-request-size متسابش على الافتراضي بتاعه وهو 10MB. صورة 12MB رجعت 500.

17) صفحات (Routes) ناقصة في Angular ✓

- أيقونة التفاصيل (i) بتروح books/details/:id ومفيش route ليها، وال BookDetailsComponent نفسه فاضي ("!works").
- لينك "My waiting list" ال route بتاعه معمول comment.
- الأتنين بيطلّعوا NG04002: Cannot match any routes.
- ومفيش route ل ** (صفحة 404).

18) صفحة Manage book بتقرأ اسم غلط للأخطاء ✓

- فين:** manage-book-component.ts:85. الكود بيقرأ err.error.validationErrors، والباك يبيعت validateErrors، فبيبقى errorMsg = undefined.
- اللي بيحصل:** أول ما change detection يشتغل بعدها بيضرب TypeError: Cannot read properties of undefined (reading 'length').
- كمان:** رسائل ال validation راجعة أكواد ("100"، "101") مش كلام، فلزم mapping في الفرونت.

19) اسم "Esmael" مكتوب ثابت في ال navbar ✓

- فين:** book/components/menu-component/menu-component.html:41. أي يوزر بيدخل ييشوف "Esmael"، واتجرب بأكونت Bob.
- الحل:** الاسم يتقري من fullName اللي جوه ال JWT (تعديل 4).

20) جدول Borrowed books بيعمل track ب id الكتاب ✓

- اللي بيحصل:** لو نفس الكتاب اتسلف مرتين، Angular بيطلّع NG0955 duplicated keys.
- الحل:** BorrowedBookResponse يرجع id ال transaction، وال track يبقى بيه.

21) ال port مختلف بين الباك والفرونت

- ال dev profile شغال على 8078، والفرونت و Swagger بيكلموا 8088. فلو الباك اشتغل عادي بال dev profile، الفرونت مش هيوصله (عشان كده اتشغل ب --server.port=8088).

- ال URL مكتوب جوه ملف generated (services/api-configuration.ts). الأحسن يبقى في environment.ts ويتمرر بـ provideApiConfiguration(...).

22) ال Feedback في صفحة Borrowed books يضيع بصمت

- الميثود giveFeedback() من غير await ، فأني غلط بيضيع.
- ال slider يبدأ من 0، والباك رافض ال 0 (@Positive). فلو الكتاب اترجع "مع تقييم" من غير ما ال slider يتحرك، التقييم مش بيتسجل ومحدث يقول.
- ال feedbackRequest مش بيتفضى بعد الإرسال.

جودة كود وترتيب

دي مش بتوقع المشروع، بس ال interviewer ممكن يلاحظها:

ال Git وحجم الريبو:

- صور يوزرز حقيقية مرفوعة على git: book-network/uploads/... فيها 12 صورة، ولازم فولدر uploads/ يتضاف لـ .gitignore.
- وفي Docker الصور بتضيع لما ال container يتمسح، لأن مفيش volume ليها.

ال Tests:

- الأمر ng test : 5 من 18 test يفشلوا ✓، لأنهم ال specs الافتراضية ومحدث عدّلها.
- الباك: ال 13 ناجحين ✓، بس مفيش tests لـ BookService ولا لـ security.

الأداء وال Transactions:

- مفيش @Transactional في BookService ، والكود شغال بس بسبب open-in-view (واللوج بيطلع warning عليه).
- فيه N+1 queries في لسته الكتب.
- كل الأغلفة بتتفري من الديسك وتتبع Base64 جوه JSON اللسته، وده ثقيل جدًا.
- الإعداد dd1-auto: update حتى في prod، والأحسن Flyway أو Liquibase.

أخطاء صغيرة في الباك:

- ملف Book.java يعمل import لـ java.beans.Transient بدل jakarta.persistence.Transient ، وفيه import متكرر.
- أسماء فيها typo:
 - autherName (ودي بقت اسم عمود في الداتابيز).
 - totalElments (ودي في ال API contract نفسه).
 - photoes-output-path
 - "Ivalid Token"
 - "User Not Found some3a"
- ال @Valid على parameters في ال services ملوش أي تأثير من غير @Validated على الكلاس.
- ال BusinessErrorCodes مكتوب فيه FORBIDDEN ، والكود بيرجع 401.

- ال JSON راجع فيه "BusinessErrorDescription" بحرف كبير جنب businessErrorCode بحرف صغير.
- ال FileStorageService :
- ييثق في ال Content-Type اللي جاي من العميل، ولو كان null ييحصل NPE.
- الامتداد يياخده من اسم الملف.
- الصور القديمة مش بتتمسح.

أخطاء صغيرة في الفرونت:

- فيه private @Output() في BookCardComponent .
- فيه import مش مستخدم من @angular/compiler في ال guard.
- بقايا SSR (@angular/ssr و express) وال SSR نفسه مش متفعل.
- فيه console.log كثير.
- كود ال pagination متكرر في 4 components.
- فيه "type="email" في خانة اسم المؤلف، و placeholder مكتوب فيه "Main St 1234".
- الصورة الافتراضية جاية من source.unsplash.com ، ودي خدمة متوقفة.
- البحث في ال navbar مش شغال، وأيقونة "waiting list" مش بتعمل حاجة.
- حجم ال bundle الأولي 566kB وده أكبر من ال budget (500kB) ✓، بسبب Bootstrap و FontAwesome كاملين.

ال CI وال Docker:

- ال CI ييشغل يدوي بس (workflow_dispatch)، ومفيش pipeline للفرونت.
- ال Dockerfile يبني بـ maven:3.8.7-openjdk-18 (نسخة مش LTS)، ويشتغل كـ root.

مدة ال JWT:

- القيمة 8640000 ms يعني 2.4 ساعة ✓. غالبًا المقصود كان يوم، وده 86400000 .
- ومفيش refresh token.

3. أسئلة الباك إند (Spring Boot)

★ = سؤال متوقع جدًا.

الأسئلة مكتوبة بالإنجليزي زي ما غالبًا هتتسأل، والإجابة بالعربي ومعها الكلمات المهمة بالإنجليزي عشان تتقال في الإنترفيو.

3.1 معمارية المشروع و Spring Boot

★ Q1. Walk me through the backend architecture. What happens when a request comes in?

المشروع layered ومتقسم **package-by-feature** (`user` , `history` , `feedback` , `book` , `auth` ...). الطلب بيعدّي الأول على الـ `Security filter chain`، وفيها `JwtAuthFilter`. بعدين بيروح لـ **Controller** (زي `BookController`)، وبعده **Service** وفيه البيزنس رولز، وبعده **Repository** (Spring Data JPA)، وآخر حاجة PostgreSQL. الداتا بتدخل وتطلع كـ **DTOs** (`BookRequest` و `record` و `BookResponse`)، وفيه **Mapper** يحوّل. الأخطاء بتتجمع في `GlobalExceptionHandler`. وكل الـ URLs تحت `/api/v1` (الـ `context-path`).

Q2. What does `@SpringBootApplication` do?

هي 3 annotations في واحدة: `@Configuration` و `@EnableAutoConfiguration` و `@ComponentScan` على الـ `com.global.book` package وكل اللي تحته. وفي المشروع متضاف عليها `@EnableJpaAuditing(auditorAwareRef = "auditAware")` و `@EnableAsync`.

Q3. Why constructor injection with `@RequiredArgsConstructor` ?

مكتبة Lombok بتعمل constructor لكل `final` field. المميزات:

- الـ dependencies مش بتتغير (immutable).
- مينفعش الـ object يتعمل من غيرها.
- سهل تعمل unit test بـ `mocks`، وده اللي معمول في `FeedbackServiceTest` بـ `@InjectMocks`.
- مفيش `field injection` بالـ `reflection`.

Q4. How do profiles work here?

الإعداد `spring.profiles.active: ${ACTIVE_PROFILE:dev}` معناه: خذ قيمة الـ environment variable `ACTIVE_PROFILE`، ولو مش موجودة استخدم `dev`.

- ملف `application-dev.yml`، على `localhost`، على `port 8078`.
- ملف `application-prod.yml`: الـ DB والميل من environment variables (`DB_URL` و `MAILDEV_URL`)، على `port 8088`.

والـ `docker-compose` ييحت `ACTIVE_PROFILE: prod`.

Q5. What is the `CommandLineRunner` bean in `BookNetworkApplication` for?

يشتغل مرة واحدة بعد ما التطبيق يقوم، ويضيف role اسمه `USER` لو مش موجود. من غيره التسجيل ييفشل برسالة "ROLE USER was not initialized". في المشاريع الحقيقية الأحسن تبقى Flyway migration.

Q6. Why DTOs and not return the entities directly? Why records for requests?

• DTOs:

- بتخبي حاجات زي الباسورد والعلاقات.
- بتمنع (Book ↔ Feedback) infinite recursion و `LazyInitializationException`.
- بتخلي ال API contract مستقل عن شكل الداتايز.

• Records:

- مش بتتغير (immutable) ومن غير boilerplate.
- شغالة كويس مع Bean Validation و Jackson.

★ Q7. How does `@Async` work in `EmailService`? What if sending fails?

ال `@EnableAsync` مع `@Async` بيخلو Spring يعمل proxy يشغل الميثود على thread pool، فالتسجيل يرجع فورًا من غير ما يستنى الإيميل.

- لو الميثود `void` ورمت exception، الغلط مش بيوصل للي نادى عليها. بيروح لـ `AsyncUncaughtExceptionHandler` ويكتب في اللوج بس (اتجرب ✓). وعشان كده throws `MessagingException` في ال controllers ملهاش لازمة.
- فخ: لو الميثود اتنادت من جوه نفس الكلاس (self-invocation) مش هتبقى async، لأن الاستدعاء مش بيعدي على ال proxy.

3.2 الأمان: JWT و Security

★ Q8. Explain the full authentication flow: register, activate, login, then an authenticated request.

1. **التسجيل (Register):** اليوزر بيتحفظ بـ `enabled=false`، والباسورد متشفّر بـ `BCrypt`، ومعه role `USER`. بعدين بيتعمل كود 6 أرقام بـ `SecureRandom`، صالح 15 دقيقة، وبيتبعث إيميل (Thymeleaf template).
2. **التفعيل (Activate):** `POST /auth/activate-account?token=` بتعمل `enabled=true` وتحفظ `validatedAt`.
3. **الدخول (Login):** `POST /auth/authenticate`، وبعدها `AuthenticationManager`، وبعده `DaoAuthenticationProvider`، وده بيحمل اليوزر بالإيميل ويقارن الباسورد ويتأكد إنه `enabled` و مش `locked`. بعد كده `JwtService` بيعمل token.
4. **أي طلب بعد كده:** Angular بيحط `Authorization: Bearer <token>` في ال header، و `JwtAuthFilter` بيتأكد منه ويحط اليوزر في ال `SecurityContext`، وال controller يياخده كـ `Authentication connectedUser`.

★ Q9. How does JwtAuthFilter work?

الفلتر يعمل extend لـ `OncePerRequestFilter` (يشتغل مرة واحدة لكل request)، وخطواته:

1. أي path يبدأ بـ `/auth` يبعدي من غير `check`.
 2. يقرأ ال header، ولو مش `Bearer` يسيب الطلب يكمل من غير `authentication`.
 3. الميثود `extractUsername` بتأكد من ال `signature` بالمفتاح، وبتطلع الإيميل من `sub`.
 4. بعدها `loadUserByUsername`، وبعدها `isTokenValid` (نفس اليوزر، وال token مش `expired`).
 5. يعمل `UsernamePasswordAuthenticationToken` بال `authorities`، ويحطه في `SecurityContextHolder`.
- الفلتر متسجل بـ `.addFilterBefore(jwtAuthFilter, UsernamePasswordAuthenticationFilter.class)`

★ Q10. What's inside the JWT? Is it encrypted?

- **header**: `{"alg": "HS256"}`
- **payload**: `sub` (الإيميل)، و `fullName`، و `authorities: ["USER"]`، و `iat`، و `exp`.
- **signature**: HMAC-SHA256 بال `secret`.

ال token **signed** مش **encrypted**. ال payload مجرد Base64URL، وأي حد يقدر يقرأه على `jwt.io`، فمينفعش يتحط فيه أي حاجة سرية. ال signature بتمنع إن حد يعدل فيه.

★ Q11. How long is the token valid? What happens after it expires?

القيمة `expiration: 8640000 ms` يعني 2.4 ساعة.

- بعد ما يخلص، `JWT` بيرمي `ExpiredJwtException` جوه الفلتر، والرد دلوقتي 403 فاضي. المفروض يبقى 401 (غلط 13).
- في Angular، ال `guard` بيشيك على ال `expiry` بـ `JwtHelperService.isTokenExpired` ويرجع اليوزر لل login وقت التنقل.
- مفيش `refresh token`، فاليوزر لازم يعمل login تاني.

Q12. How do you implement logout with JWT?

ال JWT stateless، فال `logout` الحالي هو إن الفرونت يسمح ال token (`MenuComponent.logout()`) بتنادي `clearToken()`. السيرفر مش يقدر "يلغي" token قبل ال `exp` إلا بحاجة من دول:

- قائمة `denylist` في Redis لحد ما ال token يخلص.
- استخدام `access token` قصير مع `refresh token` محفوظ في الداتايزز ويتلغي.
- رقم `token version` لكل يوزر في الداتايزز.

★ Q13. Why is CSRF disabled? And what does STATELESS mean?

- **CSRF**: هجوم ال CSRF معتمد إن المتصفح يبعث ال cookies لوحده. هنا ال token بيتبعث يدوي في ال `header Authorization`، فال CSRF مش بيأثر. لو ال JWT اتنقل لـ `cookie`، لازم CSRF يرجع يشتغل، أو يتستخدم `. SameSite=Strict`.

- **SessionCreationPolicy.STATELESS** : معناها إن Spring Security مش بيعمل `HttpSession` خالص، وكل request لازم يكون معاه ال token بتاعه. وده بيسهل ال horizontal scaling، لأن أي instance تقدر ترد على أي request.

Q14. How are passwords stored?

بـ `BCryptPasswordEncoder`. بيحط salt عشوائي وليه cost قابل للزيادة (الافتراضي 10)، وهو one-way hash مش تشفير، فمفيش decrypt. في ال login، `DaoAuthenticationProvider` يستخدم `passwordEncoder.matches`.

Q15. What happens at login if the account isn't activated or the password is wrong?

ال `DaoAuthenticationProvider` بيرمي واحد من دول:

- لو الأكونت لسه متفعلش: `DisabledException`.
- لو الباسوورد غلط: `BadCredentialsException`.
- لو الأكونت مقفول: `LockedException`.

وال `GlobalExceptionHandler` بيرجع **401** ومعاه `businessErrorCode` (303 أو 304 أو 302).

Q16. Why do you get 403 and not 401 when the token is missing or invalid?

انجرب ✓. السبب إن مفيش `AuthenticationEntryPoint` متعرف (مفيش `httpBasic` ولا `formLogin`)، ف Spring يستخدم `Http403ForbiddenEntryPoint`. وكمان أخطاء JWT بتطلع من الفلتر من غير ما حد يمسخها. الحل: `HttpStatusEntryPoint(UNAUTHORIZED)` مع `try/catch` لـ `JwtException`.

★ Q17. What is CORS and how is it configured here?

المتصفح بيمنع أي طلب من origin لـ origin تاني (من 4200 لـ 8088) إلا لو السيرفر سامح. ال `BeansConfig.corsFilter()` مطبوع كده:

- `http://localhost:4200`: origin
- مسموح بال credentials (`allowCredentials`).
- ال headers: `Origin` و `Content-Type` و `Accept` و `Authorization`.
- ال methods ومنها `PATCH`.

و `cors(withDefaults())` في `SecurityConfig` بياخد ال bean ده. وقبل الطلبات "الخطيرة"، المتصفح بيبعت **OPTIONS preflight**. ملحوظة: ال origin مكتوب ثابت، والمفروض يبقى configurable لكل بيئة.

★ Q18. Can user B edit user A's book?

في الكود الحالي أيوه (غلط 1، واتجرب ✓). ده اسمه **IDOR / Broken Access Control**، وهو رقم 1 في OWASP Top 10.

- **الحل**: check الملكية في ال service، وده الأحسن لأنه قريب من البيزنس.
- **أو**: `@PreAuthorize("@bookSecurity.isOwner(#bookId, authentication)")` بـ bean مخصص.

- الرد يبقى 403، أو 404 لو مش عايزين نكشف إن الكتاب موجود.

Q19. Where is the JWT secret stored? What's the risk?

مكتوب في الـ YAML ومرفوع على GitHub (غلط 3). اتعمل بيه token مزور ودخل ☒. الحل:

- يتقري من environment variable أو secret manager.
- يبقى فيه secret مختلف لكل بيئة.
- يتعمل rotate للـ secret الحالي.
- الـ secret يبقى 256 bit على الأقل لـ HS256، والحالي 32 byte فتمام.

Q20. What is @EnableMethodSecurity(securedEnabled = true) for?

بتشغل @PreAuthorize و @PostAuthorize و @Secured على الميثودز. مش مستخدمة لسه، وهتنفع لو اتضاف ADMIN. **فخ حلو:** الـ roles هنا اسمها USER من غير ROLE_، فـ hasRole('USER') **مش هتشتغل**، لأنها بتدور على ROLE_USER. الصح هنا hasAuthority('USER').

3.3 الداتابيز: JPA و Hibernate

★ Q21. Explain the entities and their relationships.

- كل User عنده كتب كتير (العلاقة اسمها Book.owner).
- كل Book ليه Feedback كتير.
- جدول BookTransactionHistory يربط User بـ Book، وده سجل الاستلاف، وفيه returned و returnApproved.
- العلاقة بين User و Role هي many-to-many، والـ join table اسمه _user_roles، والـ roles بتتحمل EAGER.
- كل Token (كود التفعيل) تبع User واحد.
- الكلاس BaseEntity معمول @MappedSuperclass، وفيه id وحقول الـ auditing، و Book و Feedback و BookTransactionHistory بيورثوا منه.

Q22. Why is the table called _user ?

لأن user كلمة محجوزة (reserved word) في PostgreSQL.

★ Q23. How does auditing work (@CreatedDate , @CreatedBy)?

محتاجة 3 حاجات:

- @EnableJpaAuditing(auditorAwareRef = "auditAware")
 - @EntityListeners(AuditingEntityListener.class) على BaseEntity
 - كلاس ApplicationAuditAware بيقرأ اليوزر من الـ SecurityContext
- والتواريخ بتتملى لوحدها. createdBy بيتحفظ فيه الإيميل، وبيتستخدم في FeedbackMapper عشان يعرف ownFeedback. **عيب بسيط:** لو اليوزر غيّر إيميله، الربط ده هيبوظ، والأحسن يتحفظ الـ id.

★ Q24. LAZY vs EAGER, and is there an N+1 problem?

- **الافتراضي:** @ManyToOne يبقى EAGER، و @OneToMany يبقى LAZY.
- **في المشروع:** EAGER Book.owner، و EAGER User.roles صريح، و LAZY Book.feedbacks.
- **فيه N+1:** لستة 10 كتب بتعمل query للكتب و query لا count. وبعدين لكل كتاب: query لا owner و roles بتاعه، و query لا feedbacks عشان يحسب ال rate.
- **الحل:** واحد من دول:
 - JOIN FETCH أو @EntityGraph(attributePaths = "owner")
 - @BatchSize
 - استخدام DTO projection بتحسب AVG(f.note) ب GROUP BY في query واحدة.

Q25. What is open-in-view? Why does lazy loading work in the mapper without @Transactional ?

- الإعداد spring.jpa.open-in-view شغال افتراضياً، ومعناه إن ال EntityManager ييفضل مفتوح طول ال request، فال lazy loading بيشتغل حتى في ال mapper. عيوبه:
- ال DB connection ييفضل ماسك طول الطلب.
 - ال queries بتتعمل في أماكن مش متوقعة.
- واللوج بيطلع warning عليه. الأحسن يتقفل، ويتستخدم @Transactional(readOnly = true) في ال services.

★ Q26. Where is @Transactional used? Where is it missing? What about rollback?

- **مستخدم في:** FeedbackService.save بس. كل ميثود في Spring Data repository هي transaction لوحدها.
- **ناقص في:** العمليات اللي فيها أكثر من خطوة زي borrowBook، ودي محتاجة transaction مع lock.
- **ال rollback الافتراضي:** يحصل مع RuntimeException، ومش يحصل مع checked exceptions زي MessagingException إلا لو انكتب rollbackFor.
- **فخ حلو:** لو @Transactional اتخط على activateAccount، ال branch بتاع الكود المنتهي هيحفظ كود جديد وبيعته بالإيميل، وبعدين يرمي RuntimeException. ال rollback هيمنسح الكود الجديد، مع إن الإيميل اتبعت خلاص.

★ Q27. Is there a concurrency problem when borrowing?

أيوه (غلط 4، واتجرب ✓): 4 من 5 مرات اتسلف مرتين). السبب إنه check ثم act من غير lock.

- **قفل pessimistic:** SELECT ... FOR UPDATE جوه @Transactional.
- **قفل optimistic:** @Version، ولو حصل تعارض يترمي OptimisticLockException ويتعمل retry أو يرجع 409.
- **حماية على مستوى الداتابيز:** unique partial index.

Q28. How is pagination implemented?

يتعمل ((PageRequest.of(page, size, Sort.by("createdDate").descending()))، والنتيجة Page<T> بتتحول ل PageResponse.

- ال Page يعمل count query. لو مش محتاجين العدد الكلي، Slice أسرع.
- ال sort مهم عشان الصفحات تبقى ثابتة، وفي findAllFeedbacksByBook مفيش sort.
- وفيه bug في رقم الصفحة (غلط 10).

Q29. JPQL @Query vs derived queries vs Specification: examples from the project?

- ال **Derived query**: findByEmail ، والاسم نفسه هو ال query.
- ال **JPQL**: findAllDisplayedBooks ، وده أوضح لما الشروط تبقى كتير.
- ال **Specification**: BookSpecification.withOwnerId ، مع JpaSpecificationExecutor ، ودي أحسن حاجة للفلاتر الديناميكية زي البحث.

Q30. (Trick) How do :bookId parameters work in @Query without @Param ?

لأن spring-boot-starter-parent ييشغل ال compiler flag -parameters ، فأسماء ال parameters بتت حفظ في ال .bytecode

Q31. How are IDs generated? Why is there an uploads folder user/154 ?

ال @GeneratedValue (AUTO) في PostgreSQL معناها sequence لكل entity، وده اتشاف في الداتايز: book_seq بـ increment 50. وال Hibernate ييجز 50 رقم مرة واحدة (pooled optimizer)، فبعد كل الأرقام ممكن تقفز 50.

Q32. Is ddl-auto: update OK for production?

لأ. مش ييمسح أعمدة ولا بيعمل rename، ومفيش history للتغييرات، وممكن يعمل حاجة مش متوقعة. الصح: validate مع Flyway أو Liquibase migrations متسجلة في git.

Q33. How do you prevent duplicate emails?

ال @Column(unique = true) موجود، وده constraint في الداتايز، بس بيطلع 500 (غلط 11). الحل: check بـ findByEmail قبل الحفظ، ويرجع 409. ولازم كمان يتمسك DataIntegrityViolationException ، عشان لو طلبين وصلوا في نفس اللحظة الاتنين هيعدوا ال check.

Q34. Why @SuperBuilder and not @Data on entities?

- ال @SuperBuilder : عشان ال builder بتاع Book يشمل حقول BaseEntity .
- استخدام @Data على entity فكرة وحشة، لأن equals و hashCode و toString بتلف على كل الحقول:
 - ممكن تحمّل علاقات lazy من غير قصد.
 - ممكن تعمل StackOverflow بسبب العلاقات اللي بتشاور على بعض.
- المشروع مستخدم @Getter و @Setter ، بس، وده صح.

3.4 تصميم ال API وال Errors

★ Q35. How does the global exception handler work? What would you improve?

ال `@RestControllerAdvice` فيه `@ExceptionHandler` لكل نوع، ويرجع `ExceptionResponse` ، وفيه `businessErrorCode` من ال `enum BusinessErrorCodes` . المشاكل (غلط 12، اتجرب ✓):

- ال catch-all بيحول 404 و 400 ل 500.
- ييكشف رسائل داخلية.
- يستخدم `printStackTrace` .

التحسين:

- يتعمل handler مخصوص لكل نوع.
- استخدام `ProblemDetail` (RFC 9457).
- يتعمل logging صح.

Q36. How does validation work? Why are the messages numbers like "100" ?

ال `@Valid @RequestBody` مع `constraints` على ال record. لو في غلط بيترمي `MethodArgumentNotValidException` ، والرد بيبقى 400 وفيه `{"validateErrors":["100","101",...]}` . الأكواد كانت متعمولة عشان الفروننت يترجمها (i18n)، بس الفروننت مش بيعمل ده. ملاحظات:

- ال `@NotEmpty` بيشمل `@NotNull` أصلاً.
- ال `@NotBlank` أحسن للنصوص.

Q37. Is this API RESTful? What would you change?

الكويس:

- ال URLs أسماء (nouns).
- ال pagination بـ query params.
- استخدام `PATCH` للتعديلات الجزئية.
- كود 201 للتسجيل.

اللي محتاج يتغير:

- ال `POST /books` بيعمل `create` و `update` الاثنين، والأحسن `PUT /books/{id}` للتعديل.
- ال `create` يرجع 201 ومعه `Location` header.
- ال `toggles` (`PATCH /books/shareable/{id}`) مش `idempotent`، والأحسن `PATCH /books/{id}` ومعه `body {"shareable": true}`.
- ال `upload` يرجع 202، ودي معناها "هيتعمل بعدين"، والأصح 200 أو 204.
- كلمة `totalElements` فيها typo جوه ال `contract`.

★ Q38. Explain the borrow, return, approve flow.

كل عملية استلاف لها صف في BookTransactionHistory :

```
borrow → returned=false, returnApproved=false (الكتاب مع المستلف)
return → returned=true (المستلف رجع)
approve → returnApproved=true (المالك أكد، والكتاب بقي متاح ثاني)
```

وال isAlreadyBorrowed يدور على أي صف فيه returnApproved=false .

• الشروط:

- مينفعش حد يستلف كتابه.
- الكتاب لازم يبقى shareable ومش archived.
- الكتاب مش متسلف لحد ثاني.
- العيوب: ال archive وهو متسلف (غلط 9)، وال race (غلط 4).

Q39. How does file upload work? What would you change?

ال endpoint بياخد MultipartFile @RequestParam("file") مع consumes = multipart/form-data . الصورة بتتحفظ على الديسك في {ext} . /uploads/user/{id}/{timestamp} ، وال path بيتحفظ في الداتايز. ولما حد يطلب الكتاب، الصورة بتتقري وتتبعته byte[] ، يعني Base64 جوه ال JSON. المشاكل:

- الحد الحقيقي 10MB (غلط 16).
- مفيش check ملكية (غلط 2).
- ييثق في ال content-type.
- ال Base64 ثقيل في السته.
- الديسك المحلي مش بيعمل scale، وال container بيضيع الصور.
- الأحسن: S3 أو MinIO، وبترجع URL للصورة، ولو فيه CDN يبقى أحسن.

Q40. What is springdoc / Swagger used for here?

ال springdoc بيولد /v3/api-docs و swagger-ui لوحده. @SecurityScheme(name = "bearerAuth") بيخلي ممكن تحط ال JWT في Swagger UI وتجرب. والفرونات بيولد ال client بتاعه من ال openapi.json ده بـ ng-openapi-gen. في prod، الأحسن Swagger يتقفل أو يتحمي.

3.5 الإيميل والتفعيل

Q41. Explain account activation. What are its weaknesses?

- **الفلو:** كود 6 أرقام بـ SecureRandom ، صالح 15 دقيقة، محفوظ في جدول token . لو الكود انتهى، بيتبعته كود جديد وييرجع error.
- **ليه SecureRandom مش Random :** Random ممكن حد يتوقع الأرقام اللي جاية منه.
- **العيوب (غلط 15):**

- مفيش rate limit، فمممكن تجربة كل الاحتمالات.
- الكود مش unique، وممكن يتكرر بين يوزرز.
- مش مربوط بالإيميل.
- ممكن يتستخدم ثاني ☒.
- مفيش resend.

Q42. How is the email built and tested locally?

الإيميل معمول بـ Thymeleaf template في `templates/activate_account.html` بيطلع `SpringTemplateEngine.process(name, context)` HTML، و `MimeMessageHelper` بيعته. في الـ dev فيه **MailDev** في `docker-compose`: الـ SMTP على 1025، والـ UI على `localhost:1080` ويعرض الإيميلات اللي اتبعته.

3.6 الاختبارات والـ DevOps

★ Q43. What tests did you write? How do you unit test a service?

• كلاس `FeedbackServiceTest`:

- `@ExtendWith(MockitoExtension.class)`
- الـ `@Mock` للـ `repositories`، و `@InjectMocks` للـ `service`
- `when(...).thenReturn(...)` و `assertThrows`
- `verify(repo, never()).save(any())` و `verifyNoInteractions`

- كلاس `EmailServiceTest`: بيستخدم `ArgumentCaptor` عشان يتأكد من الـ `subject`.
- الـ `contextLoads`: معمول `@SpringBootTest`، ومحتاج داتايز.

الـ 13 ناجين ☒. الناقص:

- اختبارات لـ `BookService`.
- اختبارات للـ `controllers` بـ `@WebMvcTest` أو `MockMvc` مع `spring-security-test`.
- اختبارات للـ `repositories` بـ `@DataJpaTest` و `Testcontainers`.
- وفي Boot 4 بيتستخدم `@MockitoBean`، مش `@MockBean` القديمة.

★ Q44. Explain the Dockerfile and docker-compose.

• الـ `Dockerfile (multi-stage)`:

- المرحلة الأولى Maven يبني الـ `jar`. يعمل `cache layer` للـ `dependencies` بـ `dependency:go-offline`.
- المرحلة الثانية `amazoncorretto:17` فيها الـ `jar` بس، فالـ `image` بتبقى أصغر ومن غير أدوات `build`.

• الـ `Compose`:

- خدمة `postgres` ومعها `volume`.
- خدمة `pgadmin`.
- خدمة `maildev`.

- خدمة bsn-api، ومتنظبط ليها DB_URL و MAILDEV_URL و ACTIVE_PROFILE=prod .
- خدمة bsn-api-ui (nginx).
- كلهم على نفس ال network spring-demo ، فيبكلمو بعض بالاسم (postgres:5432).
- تحسينات:
 - إضافة healthchecks مع depends_on: condition: service_healthy .
 - إضافة volume ل uploads .
 - يوزر non-root .
 - بناء ال image ب JDK 17 أو 21 .
 - ال secrets من .env .

Q45. Explain the CI pipeline.

ال GitHub Actions يمشي بالترتيب ده:

1. عمل compile.
2. تشغيل unit-tests، ومعاها Postgres ك service container.
3. عمل build (package -DskipTests).
4. عمل build-image، وده بيقرأ ال version من ال pom ويعمل push ل Docker Hub باستخدام secrets.

التحسين:

- يشتغل على push و pull_request بدل التشغيل اليدوي بس.
- إضافة cache: maven في setup-java.
- إضافة job للفروننت.
- ال build يتعمل مرة واحدة والناتج يتنقل ك artifact.

4. أسئلة الفرونت إند (Angular)

★ Q1. How is the Angular app structured and bootstrapped?

المشروع Angular 21 ب standalone components من غير NgModule . التطبيق يبدأ بـ

bootstrapApplication(App, appConfig) في app.config.ts :

- .provideRouter(routes)
- .provideHttpClient(withFetch(), withInterceptors([httpTokenInterceptor]))
- .provideBrowserGlobalErrorListeners()

القولدرات:

- فولدر pages/ فيه login و register و activate.
- فولدر book/pages/ فيه صفحات الكتب.
- فولدر book/components/ فيه الكارت والمنيو والنجوم.
- فولدر services/ فيه client ال generated و token و guard و interceptor.

★ Q2. How does routing and lazy loading work here?

- كل صفحة معمولة ب loadComponent: () => import(...) ، فكل صفحة بتبقى chunk لوحدها وبتحمل وقت ما تتطلب. ده ظاهر في ال build، مثلاً login-component حجمه 2.66kB.
- ال route books له MainComponent ك layout، فيه ال menu و <router-outlet> ، وجواه children.
- ال path الفاضي ' ' يعمل redirect ل login .
- route: ** (صفحة 404) و route التفاصيل (غلط 17).

★ Q3. How does the auth guard work? Is it real security?

- ال authGuard معمول CanActivateFn functional . يعمل inject(TokenService) ، ولو ال token مش موجود أو expired (بـ JwtHelperService) يبرّوح على login. متحط على canActivate و canActivateChild . هو مش security حقيقية، ده UX بس، لأن أي حد يقدر يعدّي الفرونت، الحماية الحقيقية في الباك. تحسين: يرجع router.createUrlTree(['/login']) بدل navigate ويرجع false .

★ Q4. How does the HTTP interceptor work?

- ال httpTokenInterceptor معمول HttpInterceptorFn functional . ال HttpRequest مينفعش يتعدّل (immutable)، فيتعامل req.clone({ setHeaders: { Authorization: 'Bearer ...' } }) وبعدين next(req) .
- تحسينات:

- ال token ميتبعتش غير لل API بس.
- إضافة catchError لو الرد 401، يعمل logout و يبرّوح login (تعديل 11).

★ Q5. Where is the token stored? Pros and cons?

في `localStorage`.

- **المميزات:** سهل، ويفضل موجود بعد الـ `refresh`.
- **العيوب:** أي JavaScript على الصفحة يقدر يقرأه، فلو حصل XSS الـ token يتسرق.
- **البديل:** cookie بـ `HttpOnly; Secure; SameSite`، ومعها CSRF protection. أو access token في الـ memory مع refresh token في cookie.

★★ Q6. What is zoneless change detection? Why do lists appear empty?

- **زمان:** `zone.js` كان يراقب أي حاجة `async` (click أو HTTP أو `setTimeout`) ويشغل change detection لوحده.
- **في Angular 21:** بقي **zoneless** افتراضياً. Angular بيعيد الرسم بس لما:
 - قيمة signal تتغير.
 - يحصل event من الـ template (زي (click)).
 - حد ينادي `markForCheck()`.
 - الـ `async` pipe يجيب قيمة جديدة.
- **في المشروع:** الـ fields العادية بتتغير بعد `await`، فالشاشة مش بتتحديث (غلط 5، اتجرب ✓): 5 كتب في الـ state و 0 على الشاشة).
- **الحل:** signals (تعديل 5).

Q7. Explain signals.

- الـ `signal(value)`: قيمة بتقراها بـ `()`، وتغيرها بـ `set()` أو `update()`.
 - الـ `computed()`: قيمة محسوبة من signals ثانية، وبتتحديث لوحدها.
 - الـ `effect()`: كود بيشغل لما signal يتغير.
 - الـ `input()` و `output()` و `model()`: البديل الجديد لـ `@Input` و `@Output`.
- الـ signals للـ state، و RxJS للـ streams والأحداث. وفيه `toSignal()` بيحول observable لـ signal.

Q8. Promises vs Observables: which does the project use?

الـ `Api.invoke` الـ generated بيحول الـ Observable لـ Promise بـ `firstValueFrom`، والكود يستخدم `async/await`.

- **ميزة الـ Promise:** بسيطة.
- **عيوبها:**
 - مفيش `cancel`. لو حد داس بسرعة على صفحات مختلفة، الردود ممكن توصل بترتيب غلط.
 - مفيش `operators`. البحث مثلاً محتاج `debounceTime` و `switchMap`.

Q9. What is ng-openapi-gen and why use it?

يولد `models` و `functions typed` من `openapi.json` بالأمر `npm run api-gen`. المميزات: `type safety`، ومفیش كود HTTP بيتكتب باليد. أي تغيير في الباك محتاج إعادة توليد. **مينفعش تعدل الملفات الـ generated**. مثلاً `rootUrl` مكتوب جوه واحد منهم، والصح إنه يتضبط بـ `provideApiConfiguration(...)`.

Q10. Template-driven vs Reactive forms?

المشروع مستخدم **template-driven** (`[(ngModel)]` مع `FormsModule`)، ومفیش `validation` في الفرونت خالص. الـ **Reactive forms** (`FormGroup` و `Validators.required/email/maxLength(8)`) أحسن للفورمز الكبيرة ولا `validation`، ويتعمل فيها نفس قواعد الباك.

★ Q11. New control flow @if/@for and why track is required.

الـ `@if` و `@else` و `@for` بدل `*ngIf` و `*ngFor`. الـ `track` إجباري عشان Angular يعرف كل عنصر، ويعيد استخدام الـ DOM بدل ما يرسم كله من الأول. لازم يكون `unique`. في `Borrowed books` الـ `track` بـ `id` الكتاب، والكتاب ممكن يتكرر، فيبطلع NG0955 (غلط 20 ✓).

Q12. How do components communicate here?

الـ `BookCardComponent` component عرض بس (presentational): يياخد `book` `@Input()` و `manage`، ويبطلع أحداث بـ `EventEmitter` `@Output()`: `share` و `archive` و `borrow` و `edit` و `details`. الصفحة الأب (smart/container) هي اللي بتكلم الـ API. في Angular الجديد ممكن `input()` و `output()` و `private` `@Output()` مش مضبوطة.

Q13. Constructor vs ngOnInit ? inject() vs constructor injection?

- الـ **constructor**: للـ DI بس.
- الـ **ngOnInit**: للـ `initialization`، زي جلب الداتا. الـ `inputs` بتكون وصلت قبله.
- الـ **inject()**: بتشتغل في الـ `field initializers`، ومهمة في الـ `functional guards` والـ `interceptors` اللي مفيهاش `constructor`. المشروع مستخدم الاتنين.
- الـ **providedIn: 'root'**: معناها `singleton` على مستوى التطبيق كله، زي `TokenService` و `Api`.

Q14. How is pagination implemented? How would you improve it?

الـ state فيه `page` و `size`، والصفحات بتتعمل بـ `Array(totalPages).fill(0).map((x, i) => i)`، ومعها أزرار `first` و `prev` و `next` و `last`. الكود ده متكرر في 4 **components**، والأحسن `PaginationComponent` واحد، يياخد `page` و `totalPages` كـ `input` ويبطلع `pageChange`.

Q15. How are images displayed and uploaded?

- العرض: Base64 في `data:image/jpg;base64,...`
- المعاينة قبل الرفع: `FileReader.readAsDataURL`
- الرفع: `multipart` بعد حفظ الكتاب. ده طلبين ورا بعض، فلو الثاني فشل الكتاب بيتحفظ من غير صورة.

Q16. How are errors shown to the user? What's wrong now?

صفحة الـ login تستخدم try/catch مع `await` ، والباقي يستخدم `.catch` ، والرسائل ستتقري من `validateErrors` أو `error` . المشاكل:

- صفحة register من غير `await` (غلط 6 ✓).
- اسم الحقل غلط في `manage-book` (غلط 18 ✓).
- حتى لو الغلط اتمسك، مش بيظهر بسبب الـ `zoneless` (غلط 5 ✓).

التحسين: interceptor مركزي للأخطاء مع `toast service`.

Q17. Angular and security (XSS)?

- الـ interpolation `{{ }}` يعمل escape لوحده، و `[innerHTML]` بيتعمل له `.sanitize`.
- الـ `bypassSecurityTrust*` خطيرة.
- الـ `guards` مش `security`.
- مفيش أي `secret` المفروض يتحط في الفرونت.

★ Q18. How is the frontend built and deployed?

الأمر `ng build` يطلع الملفات في `dist/book-network-ui/browser` ، وده جوه container فيه `nginx`. الـ `nginx.conf` فيه `try_files $uri $uri/ /index.html` . ده مهم عشان لو حد عمل `refresh` على `/books/my-books` ، `nginx` يرجع `index.html` والـ `Angular router` يكمل. الـ `budgets` في `angular.json` ، والـ `build` يطلع warning (566kB أكبر من 500kB ✓).

Q19. Why is there `isPlatformBrowser` in the guard and `TokenService`?

دي بقايا SSR. على السيرفر مفيش `localStorage` ولا `window` . بس الـ SSR مش متفعل في `angular.json` مع إن الـ `packages` متسطبة، فالأحسن يتشالوا أو الـ SSR يتفعل صح.

Q20. How do you test Angular components?

الـ `Angular 21` يستخدم **Vitest** مع `TestBed` . الـ `specs` الموجودة هي الافتراضية، و 5 من 18 ييفشلوا ✓: الـ `app.spec` مستني `<h1>Hello` ، والـ `components` اللي فيها `RouterLink` مش لاقية `ActivatedRoute` . **الحل:**

- `providers: [provideRouter([]), provideHttpClient(), provideHttpClientTesting()]`
- `HttpTestingController` عشان تعمل mock للـ API.

Q21. How would you improve performance?

- الـ `lazy loading` (موجود).
- استخدام `signals`.
- الـ `track` يبقى صح.
- مفيش `Base64` في اللستات.

- استخدام `debounce` في البحث.
- استخدام `@defer` للأجزاء الثقيلة.
- استيراد الأجزاء المطلوبة بس من Bootstrap و FontAwesome.

Q22. UX and accessibility issues?

- الأيقونات التي بتتداس هي `<i>` مش `<button>` ، فمش بتشتغل بالكيورد ومفيهاش `aria-label`.
- مفيش `loading state`.
- الأزرار مش بتتقفل وقت الطلب، فال `double click` بيعت طلبين، وده يساعد على غلط 4.
- ال `archive` مفيهوش `confirmation`.

5. لو طلبوا تعديل أو Live coding

دي أكثر تعديلات متوقعة، مرتبة من الأرجح للأقل. الأكواد مكتوبة على نفس ستايل وأسماء المشروع.

تعديل 1 — إصلاح الـ IDOR (سهل)

في `BookService.save()` جوه `if (request.id() != null)` ، وبنفس الشكل في `uploadBookCoverPicture()`:

```
book = bookRepository.findById(request.id())
    .orElseThrow(() -> new EntityNotFoundException(
        "No book found with ID:: " + request.id()));
if (!Objects.equals(book.getOwner().getId(), user.getId())) {
    throw new OperationNotPermittedException("You cannot update others books");
}
```

تعديل 2 — Delete book (متوسط)

```
// BookController
@DeleteMapping("/{book-id}")
public ResponseEntity<Void> deleteBook(@PathVariable("book-id") Integer bookId,
    Authentication connectedUser) {

    service.deleteBook(bookId, connectedUser);
    return ResponseEntity.noContent().build();
}

// BookService
@Transactional
public void deleteBook(Integer bookId, Authentication connectedUser) {
    Book book = bookRepository.findById(bookId)
        .orElseThrow(() -> new EntityNotFoundException(
            "No book found with ID:: " + bookId));
    User user = (User) connectedUser.getPrincipal();
    if (!Objects.equals(book.getOwner().getId(), user.getId())) {
        throw new OperationNotPermittedException("You cannot delete others books");
    }
    if (bookTransactionHistoryRepository.isAlreadyBorrowed(bookId)) {
        throw new OperationNotPermittedException("The book is currently borrowed");
    }
    bookRepository.delete(book);
}
```

نقطة هيسألوا فيها: الـ feedbacks والـ history فيهم foreign key على الكتاب، فالـ delete هيفشل لو فيه صفوف

مربوطة بيه. الحلول:

- تمسحهم الأول.

- أو **soft delete**، وده الأحسن. المشروع أصلاً فيه `archived` ، وممكن يتضاف `deleted` بنفس الطريقة، وال `history` يفضل لل `audit`.

وفي الفرونت: زرار في `BookCardComponent` (في وضع `manage`) عليه `(delete)` output، وقبل الحذف `confirm()`.

تعديل 3 — البحث عن كتاب بالعنوان أو المؤلف (متوسط)

ال `JpaSpecificationExecutor` موجود أصلاً:

```
// BookSpecification
public static Specification<Book> isDisplayedFor(Integer userId) {
    return (root, query, cb) -> cb.and(
        cb.IsFalse(root.get("archived")),
        cb.IsTrue(root.get("shareable")),
        cb.notEqual(root.get("owner").get("id"), userId));
}

public static Specification<Book> titleOrAuthorContains(String keyword) {
    return (root, query, cb) -> {
        String like = "%" + keyword.toLowerCase() + "%";
        return cb.or(
            cb.like(cb.lower(root.get("title")), like),
            cb.like(cb.lower(root.get("authorName")), like));
    };
}

// BookService
Page<Book> books = bookRepository.findAll(
    BookSpecification.isDisplayedFor(user.getId())
        .and(BookSpecification.titleOrAuthorContains(keyword)),
    pageable);
```

وفي ال controller: `@GetMapping("/search")` مع `@RequestParam String keyword` و `page` و `size`. في الفرونت:

1. إضافة `[(ngModel)]` على خانة البحث في ال `navbar`.
 2. استخدام `.router.navigate(['/books'], { queryParams: { q } })`.
 3. ال `BookListComponent` يسمع ل `queryParams`.
- ويستحسن `debounceTime`.

تعديل 4 — اسم اليوزر الحقيقي في ال navbar (سهل)

```
// token-service.ts
get fullName(): string {
  const token = this.token;
  return token ? this.jwtHelper.decodeToken(token)?.fullName ?? '' : '';
}

// menu-component.ts
private tokenService = inject(TokenService);
protected readonly fullName = this.tokenService.fullName;
```

```
<span class="text-capitalize fw-bold">{{ fullName }}</span>
```

تعديل 5 — إصلاح الشاشات اللي مش بتتحدث (signals) (سهل)

مثال على `BookListComponent` ، ونفس الفكرة في باقي الصفحات:

```
bookResponse = signal<PageResponseBookResponse>({});
pages = computed(() =>
  Array.from({ length: this.bookResponse().totalPages ?? 0 }, (_, i) => i));

private async findAllBooks() {
  try {
    const resp = await this.api.invoke(findAllBooks, { page: this.page, size: this.size });
    this.bookResponse.set(resp);
  } catch (err) {
    console.log(err);
  }
}
```

```
@for (book of bookResponse().content; track book.id) { ... }
@for (pageIndex of pages(); track pageIndex) { ... }
```

وأي field بيتغير بعد `await` ، زي `message` و `errorMsg` و `submitted` ، لازم يبقى signal هو كمان.

تعديل 6 — صفحة تفاصيل الكتاب مع التعليقات (متوسط)

1. إضافة route `{ path: 'details/:bookId', loadComponent: ... BookDetailsComponent }` جوه `children` ال `books`.
2. في `BookDetailsComponent` يتنادي `findBookById` و `findAllFeedbackByBook` ، والنتيجة تتحط في `signals` ، ومعاها pagination للتعليقات.
3. التعليق اللي `ownFeedback = true` يتعلم بلون مختلف.

تعديل 7 — منع تكرار الإيميل (سهل)

```
// AuthenticationService.register – قبل الحفظ
if (userRepository.findByEmail(request.getEmail()).isPresent()) {
    throw new EmailAlreadyUsedException("Email already registered");
}
```

مع handler بيرجّع 409، و code جديد في `BusinessErrorCodes`.

تعديل 8 — تغيير الباسورد (متوسط)

دي متوقعة جدًا، لأن الأكواد `INCORRECT_CURRENT_PASSWORD` (300) و `NEW_PASSWORD_DOES_NOT_MATCH` (301) موجودة في `BusinessErrorCodes` ومش مستخدمة. الخطوات:

1. إضافة endpoint `PATCH /users/password` بياخد body `{ currentPassword, newPassword, confirmationPassword }`.
2. التأكد بـ `passwordEncoder.matches(current, user.getPassword())` ، ولو غلط يرمي 300.
3. لو الجديد مش زي التأكد يرمي 301.
4. حفظ الباسورد الجديد: `user.setPassword(passwordEncoder.encode(newPassword))`.

تعديل 9 — إعادة إرسال كود التفعيل (سهل)

إضافة `POST /auth/resend-code?email=`

- لو اليوزر موجود ولسه مش `enabled` ، يتعمل كود جديد ويتبعث.
- الرد يبقى هو هو في كل الحالات، عشان محدش يعرف مين مسجل (user enumeration).
- مع rate limit.

تعديل 10 — Feedback مرة واحدة ولمن استلف بس (متوسط)

- استخدام query `existsBy...` يتأكد إن فيه transaction للكتاب ده باسم اليوزر ده، وفيها `returned = true`.
- التأكد إن مفيش feedback قبل كده من نفس اليوزر لنفس الكتاب، ومعاها unique constraint على `(created_by, book_id)`.

تعديل 11 — Interceptor يتعامل مع 401 (سهل)

```
export const httpTokenInterceptor: HttpInterceptorFn = (req, next) => {
  const tokenService = inject(TokenService);
  const router = inject(Router);
  const token = tokenService.token;
  if (token) {
    req = req.clone({ setHeaders: { Authorization: `Bearer ${token}` } });
  }
  return next(req).pipe(
    catchError((err: HttpErrorResponse) => {
      if (err.status === 401) {
        tokenService.clearToken();
        router.navigate(['/login']);
      }
      return throwError(() => err);
    })
  );
};
```

ده محتاج الباك يرجع 401 صح (غلط 13).

تعديل 12 — منع السلف المزدوج (متوسط)

```
// BookRepository
@Lock(LockModeType.PESSIMISTIC_WRITE)
@Query("SELECT b FROM Book b WHERE b.id = :id")
Optional<Book> findByIdForUpdate(@Param("id") Integer id);

// BookService
@Transactional
public Integer borrowBook(Integer bookId, Authentication connectedUser) {
    Book book = bookRepository.findByIdForUpdate(bookId)
        .orElseThrow(() -> new EntityNotFoundException(
            "No book found with ID:: " + bookId));
    // ... checks and save
}
```

الطلب الثاني بيستنى لحد ما الأول يعمل commit، وبعدها `isAlreadyBorrowed` بترجع `true` فيبترفض.

تعديل 13 — Error handling صح (سهل)

```
@ExceptionHandler(EntityNotFoundException.class)
public ResponseEntity<ExceptionResponse> handleException(EntityNotFoundException exp) {
    return ResponseEntity.status(HttpStatus.NOT_FOUND)
        .body(ExceptionResponse.builder().error(exp.getMessage()).build());
}
```

ونفس الفكرة لـ 400 و 413. والـ catch-all يرجع رسالة عامة بس، ويعمل `log.error("Unexpected error", exp)`.

تعديل 14 — Unit test لـ `borrowBook` (سهل)

```
@ExtendWith(MockitoExtension.class)
class BookServiceTest {
    @Mock private BookRepository bookRepository;
    @Mock private BookTransactionHistoryRepository bookTransactionHistoryRepository;
    @Mock private BookMapper bookMapper;
    @Mock private FileStorageService fileStorageService;
    @Mock private Authentication authentication;
    @InjectMocks private BookService bookService;

    @Test
    void shouldNotBorrowOwnBook() {
        User owner = User.builder().id(1).build();
        Book book = Book.builder().id(5).owner(owner)
            .shareable(true).archived(false).build();
        when(bookRepository.findById(5)).thenReturn(Optional.of(book));
        when(authentication.getPrincipal()).thenReturn(owner);

        assertThrows(OperationNotPermittedException.class,
            () -> bookService.borrowBook(5, authentication));
        verify(bookTransactionHistoryRepository, never()).save(any());
    }
}
```

تعديلات تانية ممكن تيجي

- **إضافة ADMIN role:**
 - إضافة role ADMIN في الـ seed.
 - عمل endpoint بـ `@PreAuthorize("hasAuthority('ADMIN')")` يعرض كل اليوزرز أو يعمل lock لأكونت.
- **قائمة الانتظار (Waiting list):**
 - عمل entity `WaitingList(user, book)` و endpoints للإضافة والحذف والعرض.
 - صفحة `my-waiting-list`.
 - إيميل لما الكتاب يرجع (bonus).
- **Refresh token:** refresh token طويل محفوظ في الداتايزز ويتلغي، و access token قصير.
- **فورم التسجيل بـ Reactive Forms:** validators زي الباك بالظبط، ورسايل تحت كل خانة.
- **الأغلفة كـ endpoint:** `URL: GET /books/{id}/cover` يرجع الصورة (image/jpeg) بدل Base64 في اللسته.
- **الترتيب (Sorting):** `sort=title,asc` في اللسته.
- **الـ API URL في environment.ts:** بدل ما يبقى ثابت في ملف generated.

6. حاجات ثانية مهمة قبل الإنترفيو

أ) أهم حاجات تتصلح الأول (تقريبًا يوم شغل)

1. إضافة check الملكية في save و uploadBookCoverPicture (غلط 1 و 2).
2. ال JWT secret ينتقل ل environment variable، ويتغير (غلط 3).
3. استخدام signals في الصفحات اللي فيها داتا async (غلط 5)، ودي أكثر حاجة هتبان لو حد شغل المشروع قدامه.
4. إضافة await في register، و shareable: false افتراضي، و @NotNull على note (غلط 6 و 7 و 8).
5. رجوع 404 و 401 صح (غلط 12 و 13).
6. إضافة @Transactional مع lock في borrowBook (غلط 4).
7. فولدر uploads/ يتشال من git ويتضاف ل .gitignore، وال port يتوحد (غلط 21).
8. ال specs اللي بتفشل تتصلح أو تتمسح (ng test).

ب) الأخطاء تتحول لقصاص (STAR)

ال interviewer يحب يسمع "لقيت مشكلة، وأثبتتها، وحليتها، واثأكدت". مثال:

"I noticed that two users could borrow the same book at the same time. I reproduced it by sending two parallel requests, and 4 out of 5 attempts created two active borrows. I fixed it with a pessimistic lock inside a transaction and added a test."

مهم: القصة دي تتقال بس بعد ما التصليح يتعمل فعلاً.

ج) Pitch في دقيقة (بالإنجليزي)

"Book Social Network is a full-stack web app for sharing books inside a community. Users register, activate their account with a 6-digit code sent by email, and log in with JWT. They can add books with a cover, choose which ones are shareable, borrow books from other members, return them, and the owner approves the return. Borrowers can rate and review books. The backend is Spring Boot 4 with Spring Security (stateless JWT), Spring Data JPA on PostgreSQL, asynchronous Thymeleaf emails and OpenAPI documentation. The frontend is Angular 21 with standalone, lazy-loaded components, a functional guard and interceptor, and a typed API client generated from the OpenAPI spec. Everything runs with Docker Compose, and a GitHub Actions pipeline compiles, tests and publishes the Docker image."

د) الأسئلة العامة اللي بتيجي مع أي مشروع

- "What was the hardest part?"

- ال security flow: ال JWT filter و CORS وال 403 مقابل 401.
- أو ال zoneless change detection في Angular 21.

- "What would you do differently?"

- كتابة tests من الأول.
- ال config يبقى بال environment variables.
- استخدام signals من البداية.
- ال error handling بـ `ProblemDetail`.
- استخدام Flyway.

- "How would you scale it?"

- ال JWT stateless، فممكّن أكثر من instance ورا load balancer.
- الصور على S3 أو MinIO مع CDN.
- إضافة indexes على `book_id` و `user_id` و `token`، لأنها مش موجودة دلوقتي.
- عمل cache للسنة الكتب (Redis).
- الإيميلات من خلال queue زي RabbitMQ أو Kafka بدل `@Async`.

- "How do you know it works?"

- ال unit tests بـ Mockito.
- ال integration tests بـ Testcontainers.
- ال Swagger للتجربة اليدوية.
- ال MailDev للإيميلات.

هـ) مصدر المشروع

هيكل المشروع قريب جدًا من الكورس المشهور **Book Social Network (BSN)** بتاع Bouali Ali: نفس الأسماء، و `BusinessErrorCodes`، و `bsn-api`، و `mail-dev-bsn`، و `spring-demo`. فيه interviewers كتير يعرفوه. لو المشروع مبني عليه، **مفيش أي مشكلة**، والأحسن يتقال ده بثقة، والتركيز يبقى على اللي اتضاف أو اتغير:

- الترقية لـ Spring Boot 4 و Angular 21 (zoneless و Vitest).
 - ال unit tests بتاعة Feedback و Email.
 - (بعد التصليح) الأخطاء اللي اتلفت واتحلت.
- المهم إن **كل سطر يبقى مفهوم وممكن يتشرح**، لأن أسئلة زي "ليه عملت كده؟" هي اللي بتفرق.

و) ال README محتاج

- صور (screenshots) أو GIF للتطبيق.
- خطوات تشغيل واضحة (`docker compose up`، وال ports، ولينك MailDev).
- رسمه architecture بسيطة.
- ملف `.env.example`.
- جزء "Known limitations / Next steps".

Login:

```
Angular —(email + password)—► POST /auth/authenticate
      —► AuthenticationManager —► DaoAuthenticationProvider
      (UserDetailsService + BCrypt)
Angular ◀— JWT (sub, fullName, authorities, exp)
```

Every request:

```
Angular interceptor (Authorization: Bearer <token>)
      —► JwtAuthFilter —► SecurityContext —► Controller
```

Borrow:

```
[available] —borrow—► [returned=false] —return—► [returned=true]
      —approve—► [available]
```

7. أرقام ومعلومات لازم تبقى محفوظة

البند	القيمة
Spring Boot / Java	4.0.6 / 17 (Hibernate 7.2)
Angular	21.2 (zoneless, standalone, Vitest)
الداتابيز	قاعدة PostgreSQL، والجداول: <code>_user</code> و <code>_user_roles</code> و <code>role</code> و <code>token</code> و <code>book</code> و <code>feedback</code> و <code>book_transaction_history</code>
Context path	<code>/api/v1</code>
Ports	API: 8088 (prod) / 8078 (dev) · UI: 4200 · MailDev: 1080 (UI) / 1025 (SMTP) · pgAdmin: 5050 · Postgres: 5432
JWT	خوارزمية HS256، ومدته 2.4 ساعة (ms 8640000)، وفيه <code>sub</code> و <code>fullName</code> و <code>authorities</code>
كود التفعيل	6 أرقام، <code>SecureRandom</code> ، صالح 15 دقيقة
Pagination	الافتراضي في الباك <code>size=10</code> ، والفرونٲ بييعت <code>size=5</code>
Upload	الإعداد (max-file-size) 50MB، بس الحد الفعلي 10MB (max-request-size)
Tests	الباك: 13 test كلهم ناجحين. الفرونٲ: 18 spec منهم 5 ييفشلوا
Endpoints	<code>auth</code> : register, authenticate, activate-account · <code>books</code> : CRUD, owner, borrowed, returned, shareable, archived, borrow, return, approve, cover · <code>feedbacks</code> : save, by book